

CI/CD Pipeline Automation – Full Project Code & Explanation

This project sets up a complete CI/CD pipeline using **Jenkins**, **Docker**, and **AWS EC2** to deploy a simple Flask app automatically.

1. Flask App (`app.py`)

```
python

from flask import Flask

app = Flask(__name__)

@app.route("/")

def home():

    return "Hello from CI/CD automated pipeline!"

if __name__ == "__main__":

    app.run(host="0.0.0.0", port=5000)
```

What it does:

- A basic web app that returns a welcome message.
- Exposes the root `/` endpoint on port 5000.

2. Dockerfile

```
Dockerfile

FROM python:3.9-slim

WORKDIR /app

COPY app.py .

RUN pip install flask

EXPOSE 5000

CMD ["python", "app.py"]
```

What it does:

- Builds a lightweight Docker image with Python and Flask
- Copies your app and runs it on container startup

3. Jenkinsfile

groovy

```
pipeline {
    agent any

    stages {
        stage('Clone Repository') {
            steps {
                git 'https://github.com/YourUsername/your-repo.git'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t flask-app .'
            }
        }

        stage('Run Docker Container') {
            steps {
                sh 'docker run -d -p 5000:5000 --name flask-app flask-app'
            }
        }
    }
}
```

What it does:

- Jenkins pulls the latest code from GitHub

- Builds a Docker image using the Dockerfile
- Runs the container and exposes the app

Notes

- Replace the Git URL in the Jenkinsfile with your actual repository.
- You can add more stages for testing, pushing to Docker Hub, or deploying to ECS.